

Deep Learning for Audio

Lecture 10

Pavel Severilov

AI Masters

2026

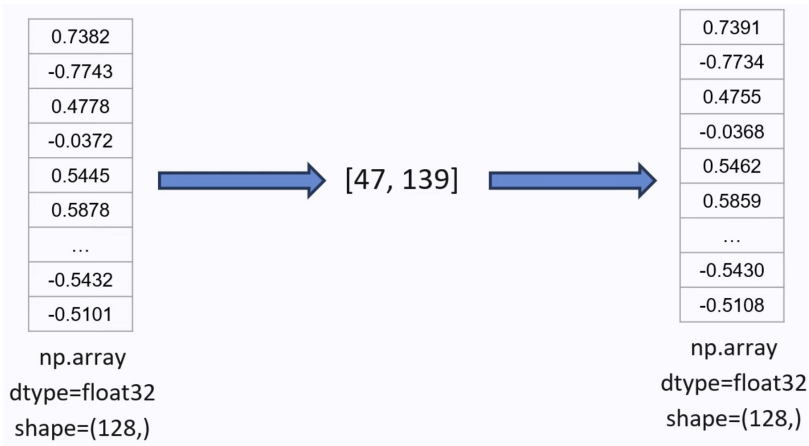
Outline

1. Discrete representation in audio: compression, quantization
2. Codec Models: SoundStream, Encodec, Mimi
3. Transformers TTS: VALL-E, Parler TTS
4. TortoiseTTS and variations (Base-TTS, Seed-TTS, Cozy-Voice)

Outline

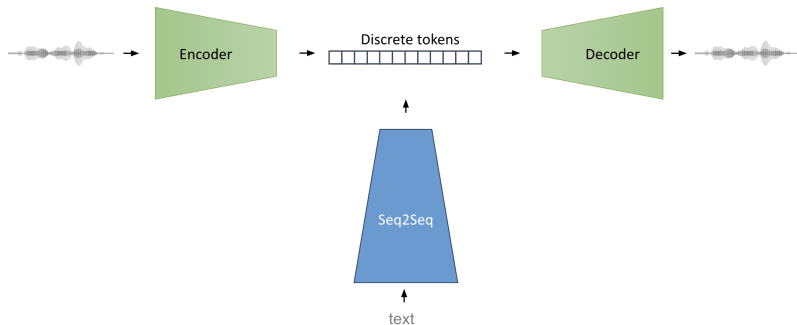
1. Discrete representation in audio: compression, quantization
2. Codec Models: SoundStream, Encodec, Mimi
3. Transformers TTS: VALL-E, Parler TTS
4. TortoiseTTS and variations (Base-TTS, Seed-TTS, Cozy-Voice)

Codec motivation



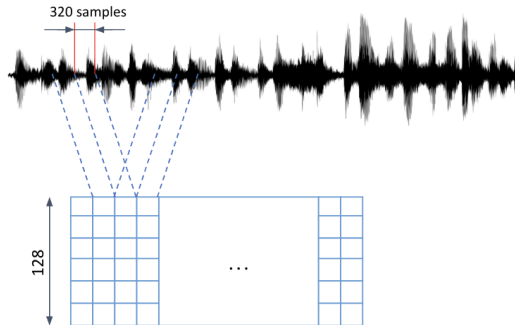
- ▶ We want to use transformers. But we can't apply them to a mel-spectrogram
- ▶ Audio vectors are continuous → We want to get discrete tokens

Base codec pipeline idea



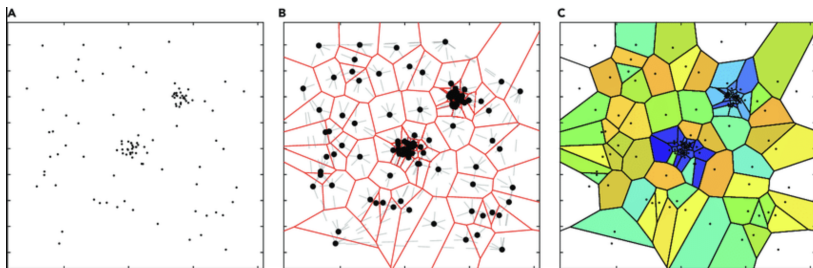
1. Encoder-decoder unsupervised training
2. Seq2seq (phonemes to tokens) supervised training

Bitrate (mp3 typical: 16-160 kbps)



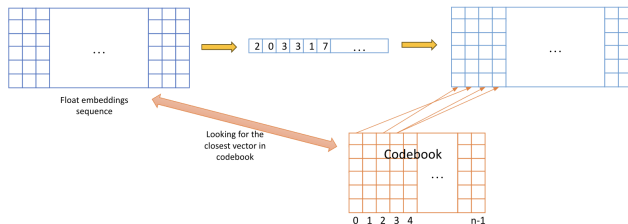
1. 1 second of audio: 24000 sample rate
2. 24000 wav samples: 320 samples per vector
3. 75 vectors: 128 floats per vector
4. 9600 floats: 16 bits per float
5. 153600 bps = 153.6 kbps

Quantization: Voronoi diagram



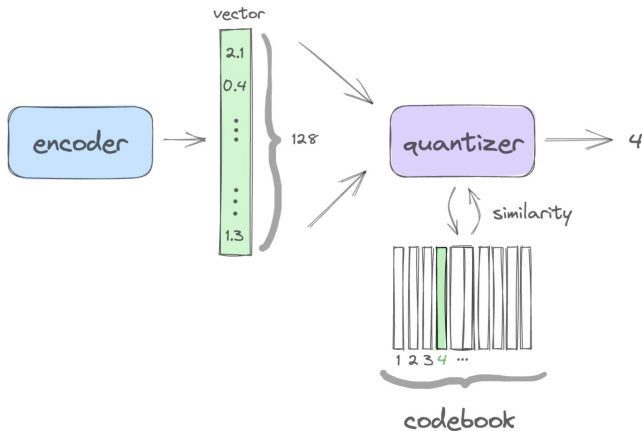
Want to compress better than mp3, AES, Opus, EVS etc. How?
Let's quantize embeddings

Vector Quantization: summary



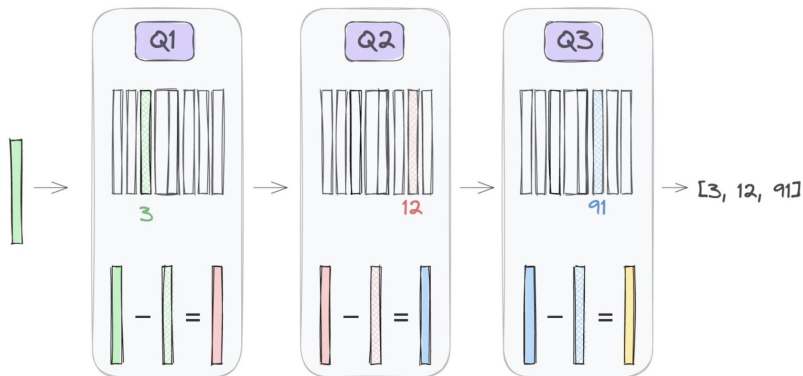
- ▶ We want to replace each vector with 1 integer
- ▶ We have a codebook. It is a set of vectors, of the same shape as vectors
- ▶ For each vector from the sequence we look for a closest one in codebook, according to L2 distance
- ▶ Replace each vector with the index of the closest vector
- ▶ Transfer the integers to the recipient
- ▶ Insert dedicated vectors, according to indices

Codebook Quantization



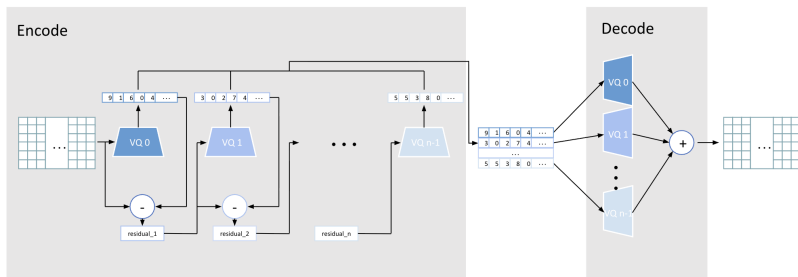
Each codebook is a vector. Number of codebook – audio token.

Residual Vector Quantization idea



- ▶ Amount of possible states equals to the size of codebook, which is few for majority of implementations
- ▶ Bad restoration quality
- ▶ Let's quantize diff between codebook and original audio embedding

Random awakening



- ▶ If a certain codebook hasn't been activated for a certain amount of batches → **Awaken it**
- ▶ Replace it with some vector of a batch
- ▶ This helps to increase the codebook coverage and codec model quality

Outline

1. Discrete representation in audio: compression, quantization
2. Codec Models: SoundStream, Encodec, Mimi
3. Transformers TTS: VALL-E, Parler TTS
4. TortoiseTTS and variations (Base-TTS, Seed-TTS, Cozy-Voice)

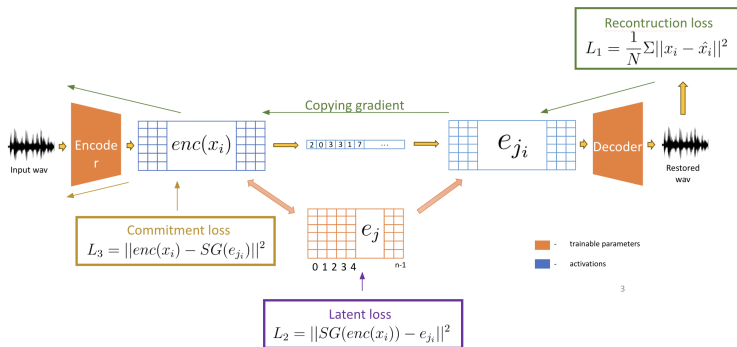
Publicly available codec models

Model	URL
Encodex [43]	https://github.com/facebookresearch/encodex
SoundStream [239]	https://github.com/weabz/SoundStream
DAC [114]	https://github.com/descriptinc/descript-audio-codec
WavTokenizer [92]	https://github.com/jishengpeng/WavTokenizer
SpeechTokenizer [250]	https://github.com/ZhangXinFD/SpeechTokenizer
SNAC [194]	https://github.com/hubertsiuzdak/snac
SemanticCodec [136]	https://github.com/haoheliu/SemanticCodec-inference
Mimi [44]	https://github.com/kyutai-labs/moshi
HiFi-Codec [231]	https://github.com/yangdongchao/AcademicCodec
FunCodec [51]	https://github.com/modelscope/FunCodec
APCodec [4]	https://github.com/YangA1520/APCodec/tree/main
AudioDec [221]	https://github.com/facebookresearch/AudioDec
FACodec [101]	https://github.com/lifeiteng/naturalaspeech3_facodec
Language-Codec [90]	https://github.com/jishengpeng/LanguageCodec
XCodec [237]	https://github.com/zhenye234/xcodec
TiCodec [178]	https://github.com/y-ren16/TiCodec
SoCodec [70]	https://github.com/bhguo/SoCodec
FUVC [254]	https://github.com/z21110008/FUVC
HILCodec [3]	https://github.com/aask1357/hilcodec
LaDiffCodec [234]	https://github.com/haiciyang/LaDiffCodec
LLM-Codec [230]	https://github.com/yangdongchao/LLM-Codec
SpatialCodec [227]	https://github.com/XZWY/SpatialCodec
BigCodec [225]	https://github.com/Aria-K-Alethia/BigCodec
SuperCodec [255]	https://github.com/exercise-book-yq/SuperCodec
RepCodec [87]	https://github.com/mct10/RepCodec
EnCodecMAE [165]	https://github.com/habla-liaa/encodema
MuCodec [226]	https://github.com/xuyaoxun/MuCodec
SPARC [32]	https://github.com/Berkeley-Speech-Group/Speech-Articulatory-Coding
BANG [173]	https://github.com/anton-jern/MULTI-AUDIODEC
SpeechRVQ [189]	https://huggingface.co/ibm/DAC_speech.v1.0
QINCo [88]	https://github.com/facebookresearch/Qinco
SimVQ [258]	https://github.com/youngsbeen/SimVQ

Differ in:

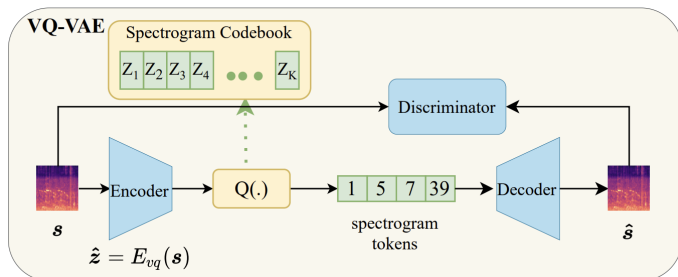
- ▶ Quantisation method (RVQ, FSQ, etc.)
- ▶ Encoder-Decoder architecture (Convs/LSTMs/Transformer blocks/etc.)
- ▶ Dataset
- ▶ Losses

How to train basic codec model



- ▶ **Reconstruction loss** (frequency domain): mel-spectrograms with diff window sizes $\rightarrow$ L1 and L2 loss
- ▶ **Discriminative loss**: k discriminators, each uses STFT with diff window length (like Hi-Fi GAN)
- ▶ **VQ commitment loss**: L2 loss, between embs from encoder and quantized values (no gradient computed for the quantized)

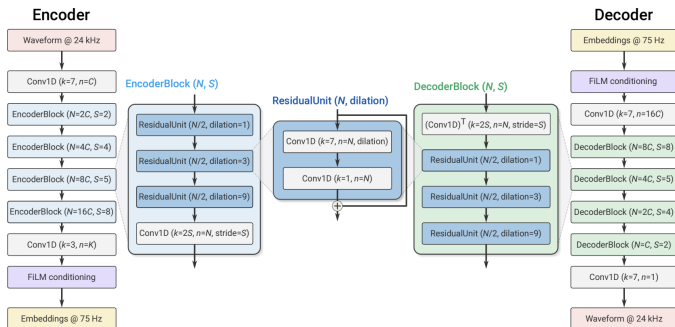
VQ-VAE (2017)



- ▶ Codebook contains a finite number of embedding vectors
- ▶ Discriminator distinguishes the mel-spectrogram is original or reconstructed
- ▶ $Q(\cdot)$: a spatial-wise quantizer that maps each features z_{ij} into its closest codebook entry z_k :

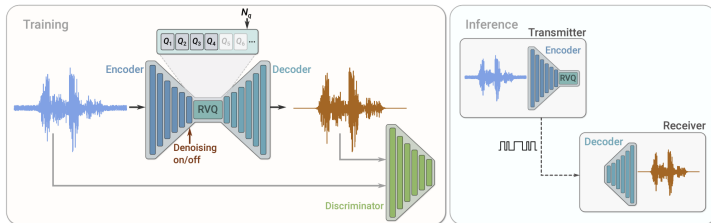
$$z_q = Q(\hat{z}) = \arg \min_{z_k \in Z} \|\hat{z}_{ij} - z_k\|_2^2$$

SoundStream (Google, 2021)



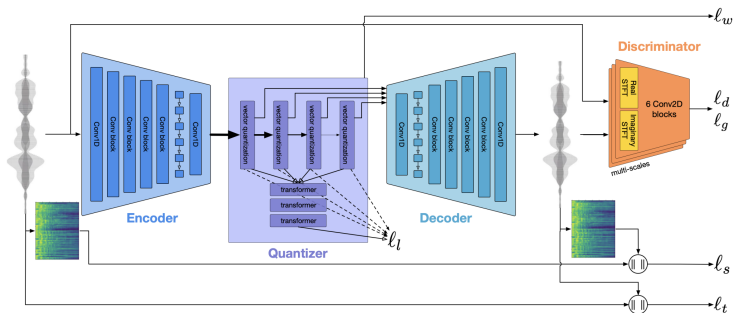
- ▶ A fully conv encoder-decoder + RVQ
- ▶ Can efficiently compress speech, music and general audio
- ▶ Outperforms SOTA codecs at 3 kbps using audio at 24 kHz sampling rate

SoundStream: train & inference



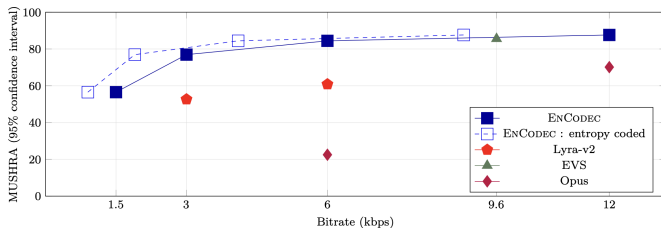
- ▶ VQ commitment loss
- ▶ L2 Reconstruction loss
- ▶ Discriminative loss
- ▶ Feature matching loss (L1 of activations from discriminator between predicted and GT sample)

Encodec (Meta, 2022)



- ▶ ConvBlocks $\rightarrow$ LSTM $\rightarrow$ Conv1D
- ▶ Train with **reconstruction + adversarial (STFT) + VQ commitment losses**
- ▶ 1 codec $\rightarrow$ 13.3ms or 75 codecs $\rightarrow$ 1s
- ▶ **Codebooks from:** clustering data + for each input updates using an exponential moving average with a decay of 0.99

Encoder: results for different bitrates



RVQ Iterations	2	4	8	16	32
Bitrate	1.5 kbps	3 kbps	6 kbps	12 kbps	24 kbps

Example for 2 RVQ iterations:

- ▶ 75 frames (audio vectors) per second
- ▶ Vector = 2 codebook indices
- ▶ 1024 codebook entries = 10 bits per entry
- ▶ $75 \times 2 \times 10 = 1500$ bits per second

Two approaches to Neural Codecs

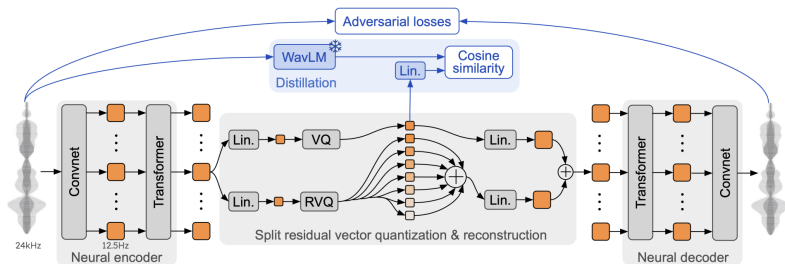
Acoustic

- ▶ Reconstruction loss
- ▶ Small and fast encoder-decoder model (Wavtokenizer, 71M parameters)
- ▶ Absorb features needed for good audio restoration
- ▶ High bitrate (i.e. wavtokenizer 75 tokens/sec)

Semantic

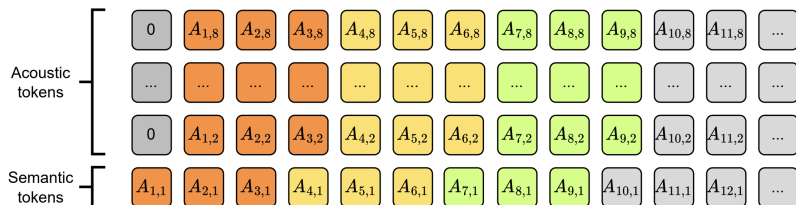
- ▶ Masked language modeling loss
- ▶ Big transformer, without decoder (w2v-BERT, 0.6B parameters)
- ▶ Absorb linguistic and semantic information
- ▶ Low bitrate (i.e. w2v-BERT 25 tokens per second)

Mimi codec (from Moshi)



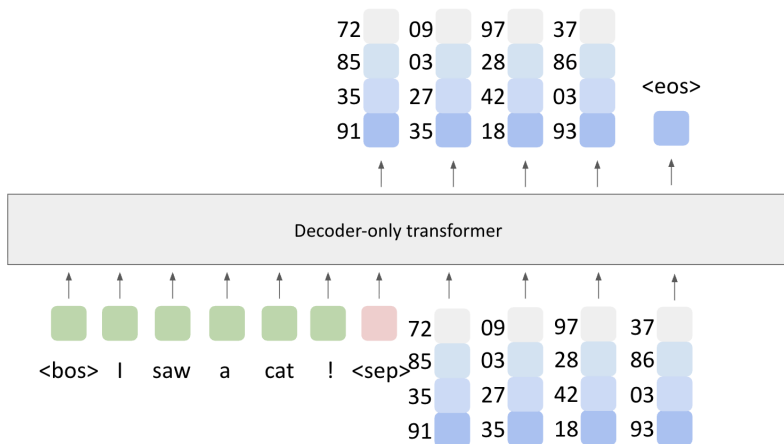
- ▶ Similar to Encodec: add Transformer to conv blocks in encoder and decoder + remove reconstruction loss
- ▶ Modeling semantic and acoustic information with a single model: add VQ using CosSim with WavLM
- ▶ Distill WavLM's semantic tokens into the first layer of acoustic tokens

How to generate RVQ tokens



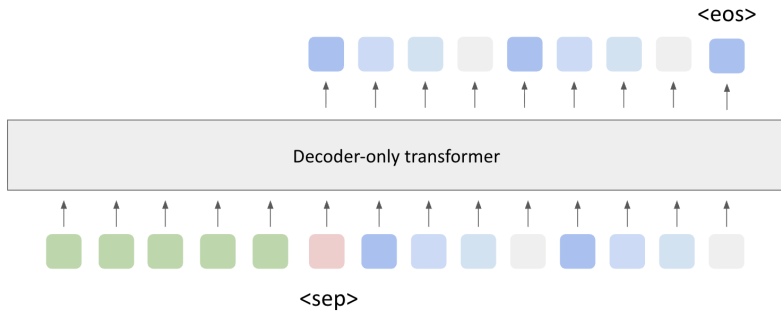
- ▶ Generate one by one in a linear fashion (very long, too many steps, especially if there are multiple layers)
- ▶ Generate in parallel (lower quality, the tokens are dependent on each other)
- ▶ Stacked delay (a compromise between sequential and parallel generation)

RVQ tokens in transformers



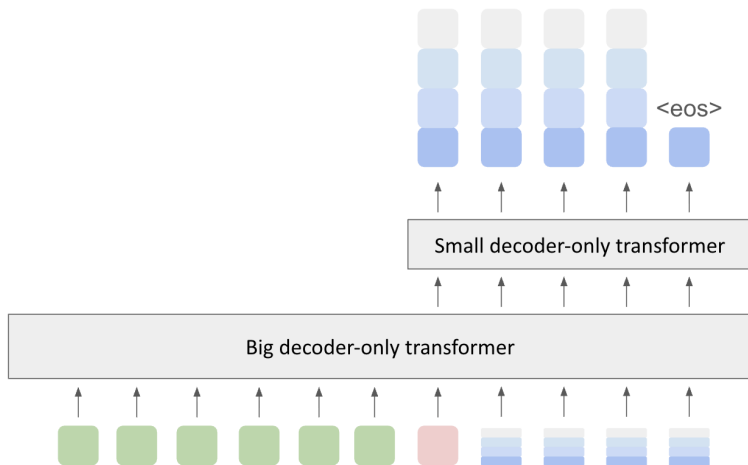
How to make N tokens per step?

RVQ tokens in transformers: AudioLM idea



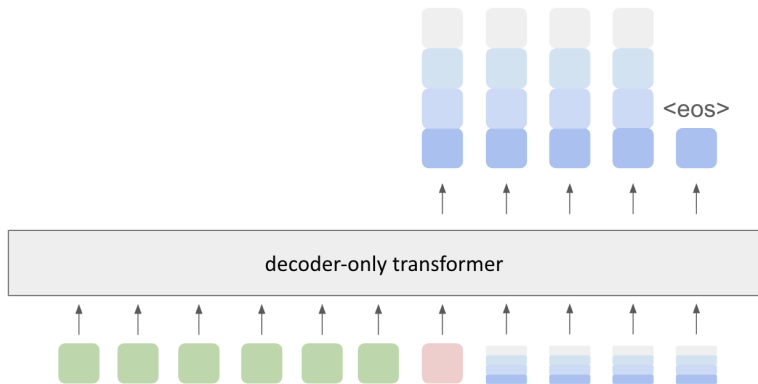
Just flatten? A lot of compute

RVQ tokens in transformers: MQTTS idea (two transformers)



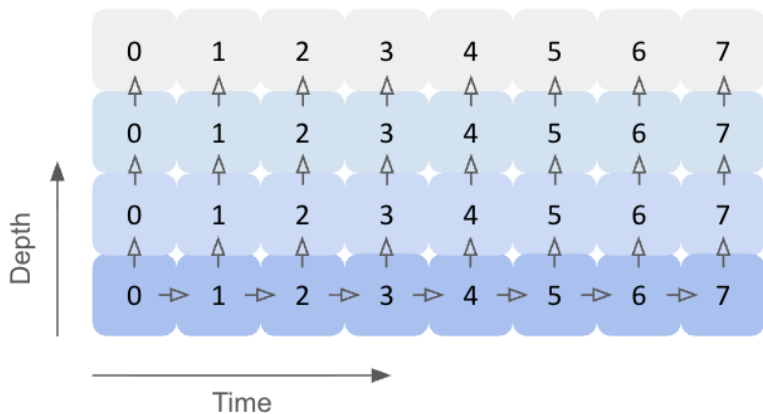
Second transformer? It complicates the model

RVQ tokens in transformers: AudioGen idea

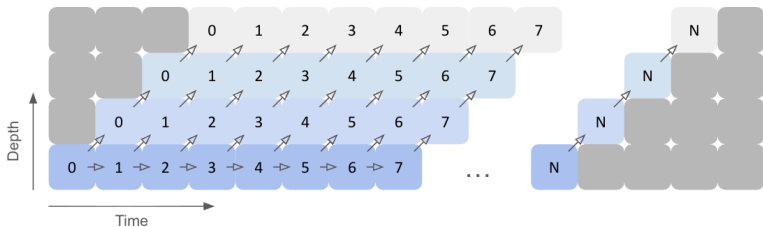


Can we just take N linear layers?

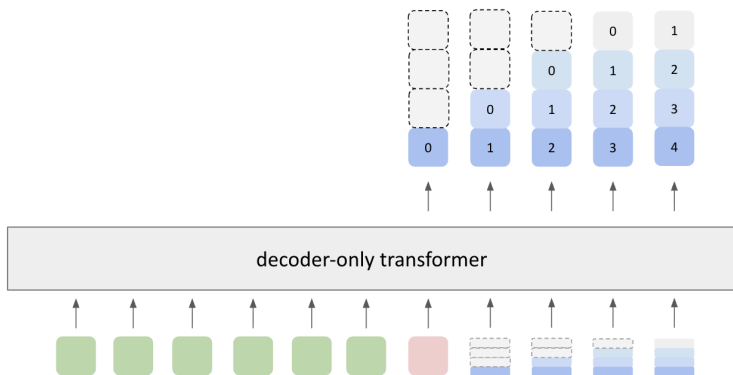
RVQ tokens: linear generation



RVQ tokens: stacked delay idea



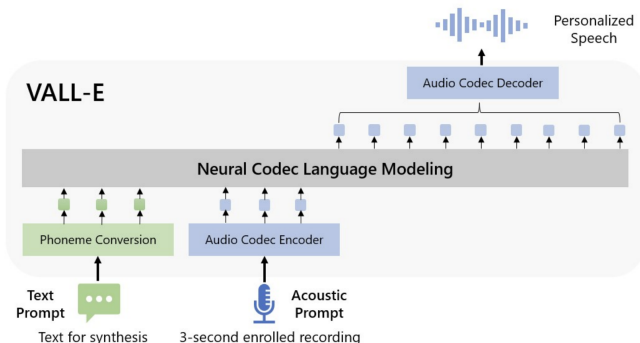
RVQ tokens: stacked delay idea



Outline

1. Discrete representation in audio: compression, quantization
2. Codec Models: SoundStream, Encodec, Mimi
3. Transformers TTS: VALL-E, Parler TTS
4. TortoiseTTS and variations (Base-TTS, Seed-TTS, Cozy-Voice)

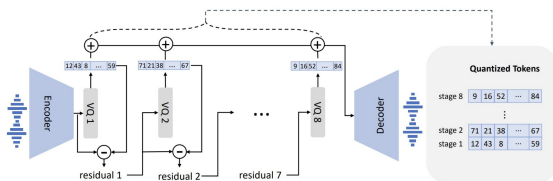
VALL-E: zero-shot TTS (Microsoft, 2023)



- ▶ Encodec + transformer layers for Codec Language Modeling
- ▶ Codec: EnCodec with 8 layers (1second = 75×8 tokens)
- ▶ Data: LibriLight, 60000h
- ▶ N tokens per step solution: Two transformers

Wang et al., *Neural Codec Language Models are Zero-Shot Text to Speech Synthesizers*, Microsoft, 2023

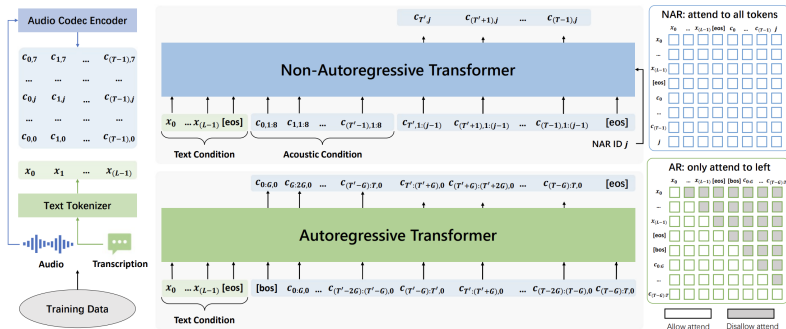
VALL-E: RVQ



- ▶ The first quantizer plays the most important role in reconstruction
- ▶ **Split generation of 8 tokens** from quantizers into **2 parts**: Autoregressive Transformer (**AR**) and Non-Autoregressive (**NAR**) Transformer
- ▶ **AR** generates the **tokens from the first quantizer** (main information about audio: num of frames, tempo, timbre)
- ▶ **NAR** generates **other 7 tokens** (noise, hissing, high frequencies)

Wang et al., *Neural Codec Language Models are Zero-Shot Text to Speech Synthesizers*, Microsoft, 2023

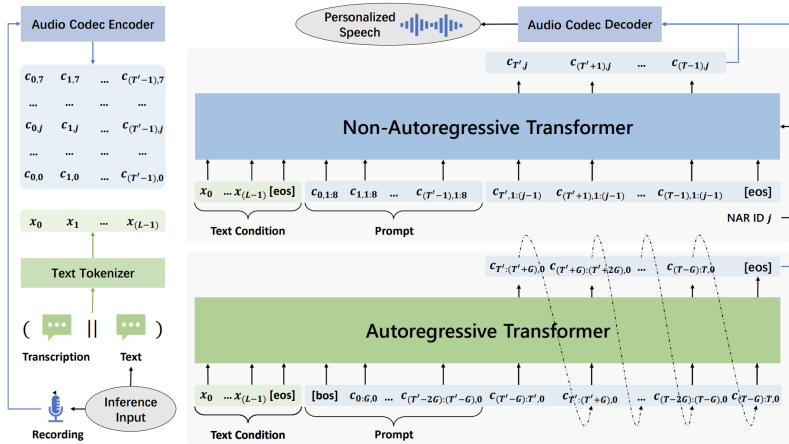
VALL-E 2 (Microsoft, 2024)



- ▶ Based on **VALL-E + more clean data** in train
- ▶ **AR generate**: Grouped Code Modeling – 4-8 codes/frame (x4-8 faster)
- ▶ **Decoding**: VALL-E – top-p, VALL-E 2 – nucleus (more stable)

Chen et al., VALL-E 2: Neural Codec Language Models are Human Parity Zero-Shot Text to Speech Synthesizers, Microsoft, 2024

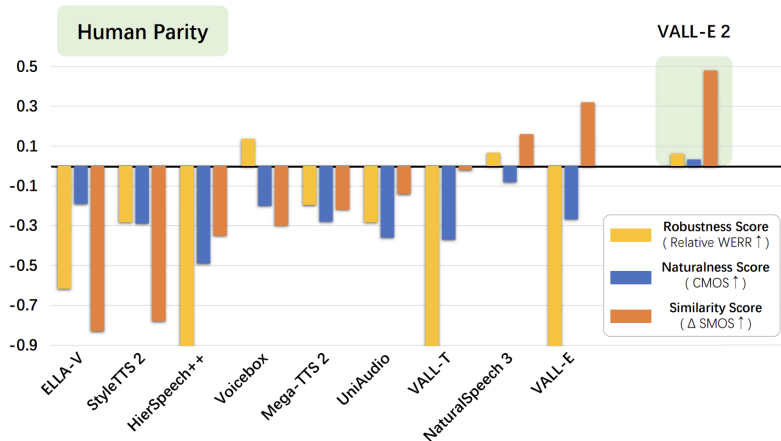
VALL-E 2: inference



- NAR decoder will be called seven times to generate codes in seven quantizers

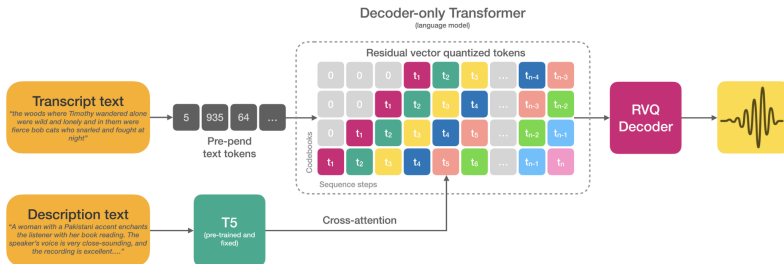
Chen et al., VALL-E 2: Neural Codec Language Models are Human Parity Zero-Shot Text to Speech Synthesizers, Microsoft, 2024

VALL-E 2: results



Chen et al., VALL-E 2: Neural Codec Language Models are Human Parity Zero-Shot Text to Speech Synthesizers, Microsoft, 2024

Parler TTS (2024)



- ▶ Data: English part of LibriSpeech, 45h
- ▶ Codec: Descript (Encodec variation) with 8 layers
- ▶ Speaker conditioning: Textual prompt + T5
- ▶ N tokens per step solution: Delayed pattern

Dan Lyth, Simon King, Natural language guidance of high-fidelity text-to-speech models with synthetic annotations, 2024

Outline

1. Discrete representation in audio: compression, quantization
2. Codec Models: SoundStream, Encodec, Mimi
3. Transformers TTS: VALL-E, Parler TTS
4. TortoiseTTS and variations (Base-TTS, Seed-TTS, Cozy-Voice)

Two approaches to TTS

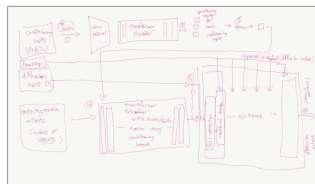
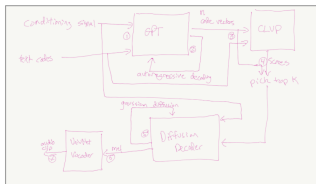
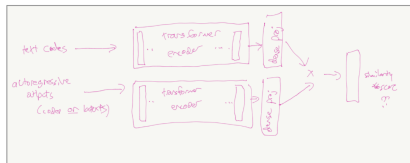
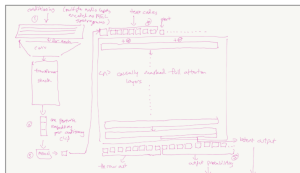
Texts like

- ▶ Discrete targets, aka tokens from codec models
- ▶ Crossentropy
- ▶ Transformers
- ▶ Autoregression + Streaming

Images like

- ▶ Continuous targets, aka good old mel-spectrogram
- ▶ MSE
- ▶ Diffusion
- ▶ No streaming

TortoiseTTS: Architectural Design Doc (2022)



TortoiseTTS: final of the story

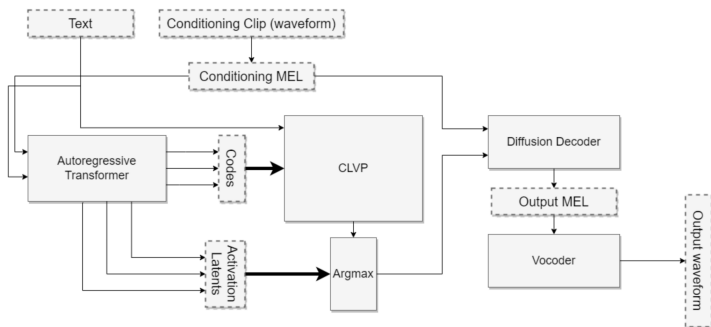
I've Joined OpenAI

Posted on October 28, 2022 by jbetker

I've been meaning to write this for a couple of months now, but simply haven't found the time. Life has gotten quite busy for me lately, and I hope to explain why. First, the elephant in the room – I have left Google and finally stepped into the ML industry. I've accepted a position as...

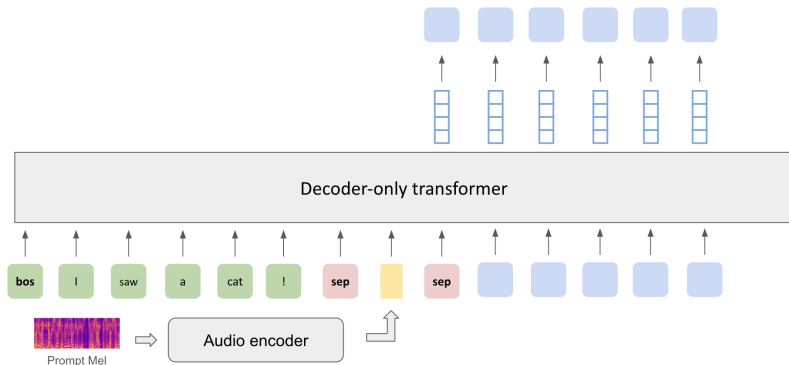
Continue reading

TortoiseTTS: architecture



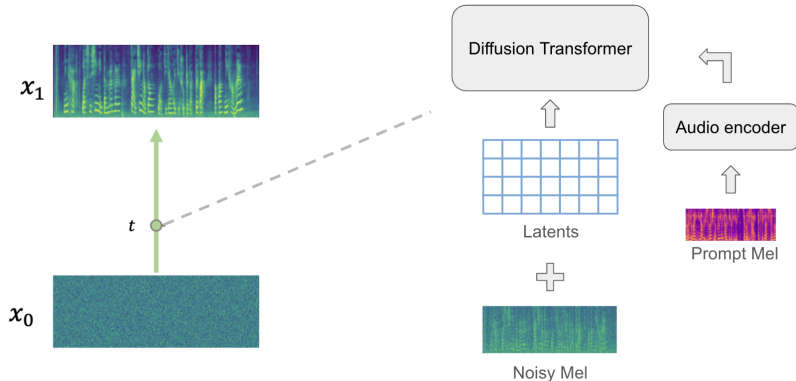
- ▶ Let's adapt ideas from image generation **Dalle-1** to **TTS** task
- ▶ Uses both **autoregressive transformer** and **diffusion**
 - ▶ **Autoregressive model** are strong for converting between unaligned domains, like vision, text and speech
 - ▶ **DDPMs** operate in a continuous domain, which allows them to model expressive modalities
 - ▶ Need **audio quantization** to use transformers

TortoiseTTS: Autoregressive GPT, take latents



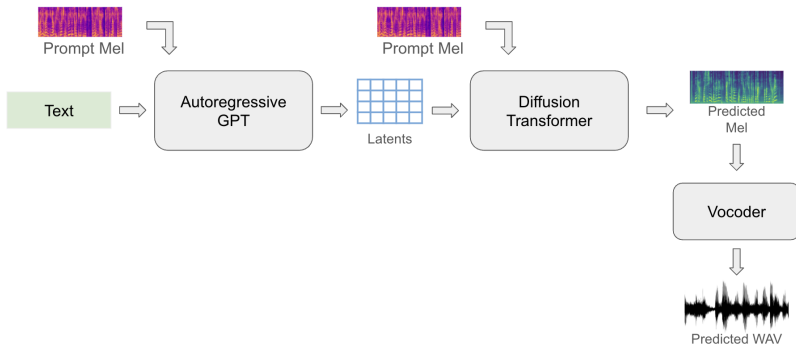
- ▶ For further generation take latents instead of tokens, as they carry more information
- ▶ Audio encoder from pretrained VQ-VAE aka codec model (1 layered codebook, 25 tokens per second)

TortoiseTTS: diffusion



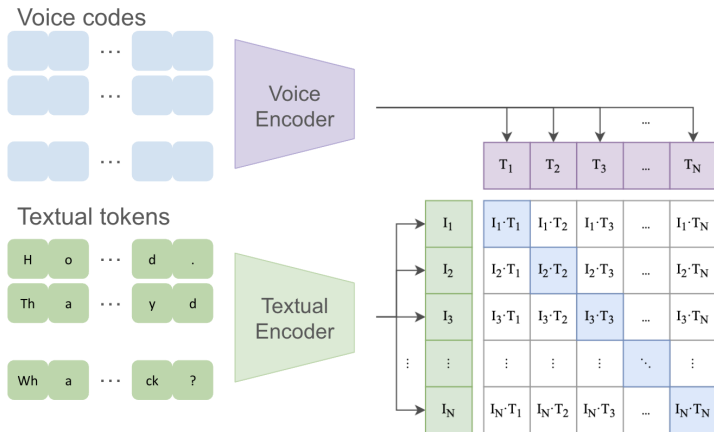
- ▶ Noise – input, latent representations + melspec (speaker style) – conditioning
- ▶ Latent representations: transformer output from previous step
- ▶ 50-100 diffusion steps (a lot)

TortoiseTTS: base scheme



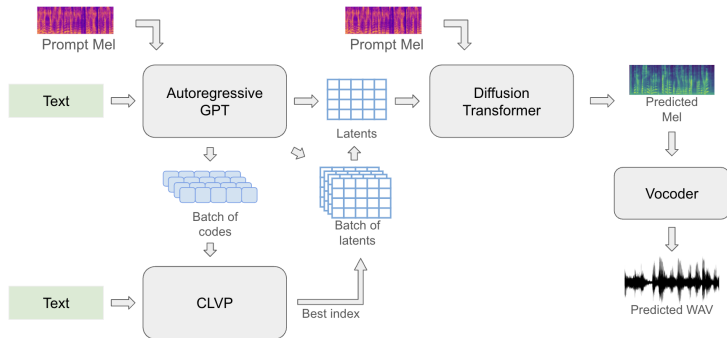
- ▶ We consider here, that diffusion + vocoder is just a powerful decoder from 25Hz to Wav
- ▶ Autoregressive GPT is the main generative power
- ▶ GPT generates the same distribution as the data, so it can generate both good and bad quality samples

TortoiseTTS: CLVP (CLIP but for Voice)



- ▶ Let's make CLIP but for Voice: embedding of text and audio with one content to be similar
- ▶ Train two transformers (contrastive loss), which represent text and audio as an embedding

TortoiseTTS: overall scheme



- ▶ Let's sample many times from GPT and choose the best latents
- ▶ CLVP can rerank multiple AR outputs without the expensive diffusion model being invoked

TortoiseTTS: problems?

1. We need to train 5 models: VQ-VAE, GPT, Diffusion, Vocoder, CLVP
2. It's not streamable (wait for all steps of GPT for passing to diffusion)
3. A lot of steps for diffusion
4. VQ-VAE is not ideal way to get audio tokens
5. CLVP seems to be a crutch: better finetune

TortoiseTTS: variations

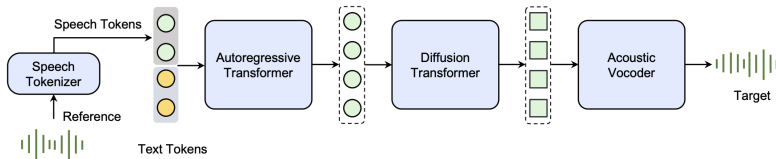


Figure: Seed-TTS inference

- ▶ **Seed-TTS (ByteDance, 2024)**
 - ▶ Alternative tokenizer (didn't mention which one)
 - ▶ Remove CLVP via fine-tuning on 5 voices
- ▶ **CosyVoice (Alibaba, 2024-2025)**
 - ▶ Replace VQ-VAE with ASR-pretrained model
 - ▶ Replace Diffusion with Flow Matching
- ▶ **Base-TTS (Amazon, 2024)**
 - ▶ Larger GPT: 300M → 1B
 - ▶ VQ-VAE → WavLM